

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №4.

Тема: «Простое наследование. Принцип подстановки»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

Определить пользовательский класс.

Определить в классе следующие конструкторы:

- без параметров (конструктор по умолчанию)
- с параметрами
- копирования

Определить в классе деструктор.

Определить в классе компоненты-функции для просмотра и установки полей данных:

- селекторы (get-методы)
- модификаторы (set-методы)

Перегрузить операцию присваивания (=).

Перегрузить операции ввода и вывода объектов с помощью потоков (<< и >>).

Определить производный класс (наследник).

Написать программу, в которой продемонстрировать:

- создание объектов
- работу всех перегруженных операций

Реализовать функции, получающие и возвращающие объект базового класса.

Продемонстрировать принцип подстановки (когда объект производного класса используется как объект базового класса).

Базовый класс: ЧЕЛОВЕК (PERSON)

Поля:

- Имя (name) — string
- Возраст (age) — int

Требуется: Определить методы изменения полей (сеттеры)

Производный класс: EMPLOYEE (СОТРУДНИК)

Наследуется от класса PERSON.

Дополнительные поля:

- Должность (position) — string
- Оклад (salary) — double

Требуется:

- Определить методы изменения полей
- Реализовать вычисление зарплаты сотрудника по формуле:
- Зарплата = Оклад + Премия (% от оклада)

Анализ задачи:

1. Создаём базовый класс Person с именем и возрастом, затем производный класс Employee с должностью и окладом.
2. Реализуем все конструкторы, деструкторы, get/set методы
3. Перегружаем операторы =, <<, >>.
4. В main() показываем создание объектов, работу операторов и принцип подстановки (когда Employee используется как Person).

Код:

```
#include <iostream>
#include <string>
using namespace std;

// БАЗОВЫЙ КЛАСС: PERSON (ЧЕЛОВЕК)
class Person {
protected: // protected, чтобы производный класс имел доступ
    string name; // Имя
    int age;      // Возраст

public:
    // КОНСТРУКТОРЫ

    // 1. Конструктор по умолчанию
    Person() : name("Неизвестно"), age(0) {
        cout << "Вызван конструктор по умолчанию Person\n";
    }

    // 2. Конструктор с параметрами
    Person(string n, int a) : name(n), age(a) {
        cout << "Вызван конструктор с параметрами Person\n";
    }
}
```

```

}

// 3. Конструктор копирования
Person(const Person& other) : name(other.name), age(other.age) {
    cout << "Вызван конструктор копирования Person\n";
}

// ДЕСТРУКТОР
~Person() {
    cout << "Вызван деструктор Person для: " << name << "\n";
}

// СЕЛЕКТОРЫ (get-методы)
string getName() const { return name; }
int getAge() const { return age; }

// МОДИФИКАТОРЫ (set-методы)
void setName(string n) { name = n; }
void setAge(int a) { age = a; }

// ПЕРЕГРУЗКА ОПЕРАТОРА ПРИСВАИВАНИЯ
Person& operator=(const Person& other) {
    if (this != &other) { // Проверка на самоприсваивание
        name = other.name;
        age = other.age;
    }
    cout << "Вызван оператор присваивания Person\n";
    return *this;
}

// ДРУЗЬЯ ДЛЯ ВВОДА/ВЫВОДА
friend ostream& operator<<(ostream& out, const Person& p);
friend istream& operator>>(istream& in, Person& p);
};

// ОПЕРАТОРЫ ВВОДА/ВЫВОДА ДЛЯ PERSON

ostream& operator<<(ostream& out, const Person& p) {

```

```

        out << "Имя: " << p.name << ", Возраст: " << p.age;
        return out;
    }

```

```

istream& operator>>(istream& in, Person& p) {
    cout << "Введите имя: ";
    in >> p.name;
    cout << "Введите возраст: ";
    in >> p.age;
    return in;
}

```

// ПРОИЗВОДНЫЙ КЛАСС: EMPLOYEE (СОТРУДНИК)

```

class Employee : public Person {

```

```

private:

```

```

    string position; // Должность

```

```

    double salary; // Оклад

```

```

public:

```

```

    // КОНСТРУКТОРЫ

```

```

    // 1. Конструктор по умолчанию

```

```

    Employee() : Person(), position("Не определена"), salary(0.0) {
        cout << "Вызван конструктор по умолчанию Employee\n";
    }

```

```

    // 2. Конструктор с параметрами

```

```

    Employee(string n, int a, string pos, double sal)
        : Person(n, a), position(pos), salary(sal) {
        cout << "Вызван конструктор с параметрами Employee\n";
    }

```

```

    // 3. Конструктор копирования

```

```

    Employee(const Employee& other)
        : Person(other), position(other.position), salary(other.salary) {
        cout << "Вызван конструктор копирования Employee\n";
    }

```

```

// ДЕСТРУКТОР
~Employee() {
    cout << "Вызван деструктор Employee для: " << name << "\n";
}

// СЕЛЕКТОРЫ
string getPosition() const { return position; }
double getSalary() const { return salary; }

// МОДИФИКАТОРЫ
void setPosition(string pos) { position = pos; }
void setSalary(double sal) { salary = sal; }

// ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ
double calculateTotalSalary(double bonusPercent) const {
    // Зарплата = Оклад + Премия (% от оклада)
    double bonus = salary * bonusPercent / 100.0;
    return salary + bonus;
}

// ПЕРЕГРУЗКА ОПЕРАТОРА ПРИСВАИВАНИЯ
Employee& operator=(const Employee& other) {
    if (this != &other) {
        Person::operator=(other); // Копируем базовую часть
        position = other.position;
        salary = other.salary;
    }
    cout << "Вызван оператор присваивания Employee\n";
    return *this;
}

// ДРУЗЬЯ ДЛЯ ВВОДА/ВЫВОДА
friend ostream& operator<<(ostream& out, const Employee& e);
friend istream& operator>>(istream& in, Employee& e);
};

// ОПЕРАТОРЫ ВВОДА/ВЫВОДА ДЛЯ EMPLOYEE

```

```
ostream& operator<<(ostream& out, const Employee& e) {
    out << "Имя: " << e.name << ", Возраст: " << e.age
        << ", Должность: " << e.position << ", Оклад: " << e.salary;
    return out;
}
```

```
istream& operator>>(istream& in, Employee& e) {
    cout << "Введите имя: ";
    in >> e.name;
    cout << "Введите возраст: ";
    in >> e.age;
    cout << "Введите должность: ";
    in >> e.position;
    cout << "Введите оклад: ";
    in >> e.salary;
    return in;
}
```

// ФУНКЦИЯ, ПРИНИМАЮЩАЯ И ВОЗВРАЩАЮЩАЯ ОБЪЕКТ БАЗОВОГО КЛАССА

```
Person createPerson(string name, int age) {
    // Создаём и возвращаем объект базового класса
    return Person(name, age);
}
```

```
void printPersonInfo(Person p) {
    // Функция принимает объект базового класса
    // Благодаря принципу подстановки, можно передать и Employee
    cout << "Информация о человеке: " << p << "\n";
}
```

```
int main() {
    setlocale(LC_ALL, "ru");

    cout << "1. КОНСТРУКТОРЫ БАЗОВОГО КЛАССА\n\n";

    Person p1; // Конструктор по умолчанию
    cout << "p1: " << p1 << "\n\n";
}
```

```

Person p2("Иван", 25); // Конструктор с параметрами
cout << "p2: " << p2 << "\n\n";

Person p3 = p2; // Конструктор копирования
cout << "p3 (копия p2): " << p3 << "\n\n";

cout << "2. КОНСТРУКТОРЫ ПРОИЗВОДНОГО КЛАССА\n\n";

Employee e1; // Конструктор по умолчанию
cout << "e1: " << e1 << "\n\n";

Employee e2("Мария", 30, "Менеджер", 50000); // С параметрами
cout << "e2: " << e2 << "\n\n";

Employee e3 = e2; // Конструктор копирования
cout << "e3 (копия e2): " << e3 << "\n\n";

cout << "3. СЕЛЕКТОРЫ И МОДИФИКАТОРЫ\n\n";

cout << "Имя e2: " << e2.getName() << "\n";
cout << "Должность e2: " << e2.getPosition() << "\n";
cout << "Оклад e2: " << e2.getSalary() << "\n\n";

e2.setName("Анна");
e2.setPosition("Старший менеджер");
e2.setSalary(60000);
cout << "После изменения e2: " << e2 << "\n\n";

cout << "4. ОПЕРАТОР ПРИСВАИВАНИЯ\n\n";

Person p4;
p4 = p2;
cout << "p4 после присваивания: " << p4 << "\n\n";

Employee e4;
e4 = e2;
cout << "e4 после присваивания: " << e4 << "\n\n";

```



```

cout << "5. ВВОД С КЛАВИАТУРЫ\n\n";

Employee e5;
cout << "Введите данные для e5:\n";
cin >> e5;
cout << "Вы ввели: " << e5 << "\n\n";

cout << "6. ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ\n\n";

cout << "e2: " << e2 << "\n";
double bonus = 15.0; // 15% премии
cout << "Премия: " << bonus << "%\n";
cout << "Итоговая зарплата: " << e2.calculateTotalSalary(bonus) << "\n\n";

cout << "e5: " << e5 << "\n";
cout << "Итоговая зарплата (с премией 20%): "
    << e5.calculateTotalSalary(20.0) << "\n\n";

cout << "7. ПРИНЦИП ПОДСТАНОВКИ\n\n";

// Функция принимает Person, но мы передаём Employee
// Это работает благодаря наследованию (принцип подстановки)
cout << "Передаём e2 (Employee) в функцию printPersonInfo:\n";
printPersonInfo(e2);
cout << "\n";

// Функция возвращает Person
cout << "Вызов функции createPerson:\n";
Person p5 = createPerson("Петр", 35);
cout << "p5: " << p5 << "\n\n";

cout << "8. ВСЕ ОБЪЕКТЫ\n\n";
cout << "p1: " << p1 << "\n";
cout << "p2: " << p2 << "\n";
cout << "p3: " << p3 << "\n";
cout << "p4: " << p4 << "\n";
cout << "p5: " << p5 << "\n";
cout << "e1: " << e1 << "\n";

```

```
cout << "e2: " << e2 << "\n";  
cout << "e3: " << e3 << "\n";  
cout << "e4: " << e4 << "\n";  
cout << "e5: " << e5 << "\n";  
  
return 0;  
}
```

Результат выполнения:

1. КОНСТРУКТОРЫ БАЗОВОГО КЛАССА

Вызван конструктор по умолчанию Person
p1: Имя: Неизвестно, Возраст: 0

Вызван конструктор с параметрами Person
p2: Имя: Иван, Возраст: 25

Вызван конструктор копирования Person
p3 (копия p2): Имя: Иван, Возраст: 25

2. КОНСТРУКТОРЫ ПРОИЗВОДНОГО КЛАССА

Вызван конструктор по умолчанию Person
Вызван конструктор по умолчанию Employee
e1: Имя: Неизвестно, Возраст: 0, Должность: Не определена, Оклад: 0

Вызван конструктор с параметрами Person
Вызван конструктор с параметрами Employee
e2: Имя: Мария, Возраст: 30, Должность: Менеджер, Оклад: 50000

Вызван конструктор копирования Person
Вызван конструктор копирования Employee
e3 (копия e2): Имя: Мария, Возраст: 30, Должность: Менеджер, Оклад: 50000

3. СЕЛЕКТОРЫ И МОДИФИКАТОРЫ

Имя e2: Мария
Должность e2: Менеджер
Оклад e2: 50000

После изменения e2: Имя: Анна, Возраст: 30, Должность: Старший менеджер, Оклад: 60000

4. ОПЕРАТОР ПРИСВАИВАНИЯ

Вызван конструктор по умолчанию Person
Вызван оператор присваивания Person
p4 после присваивания: Имя: Иван, Возраст: 25

Вызван конструктор по умолчанию Person
Вызван конструктор по умолчанию Employee
Вызван оператор присваивания Person
Вызван оператор присваивания Employee
e4 после присваивания: Имя: Анна, Возраст: 30, Должность: Старший менеджер, Оклад: 60000

5. ВВОД С КЛАВИАТУРЫ

Вызван конструктор по умолчанию Person
Вызван конструктор по умолчанию Employee
Введите данные для e5:
Введите имя: Тапуа
Введите возраст: 27
Введите должность: Cleaner
Введите оклад: 20000
Вы ввели: Имя: Тапуа, Возраст: 27, Должность: Cleaner, Оклад: 20000

6. ВЫЧИСЛЕНИЕ ЗАРПЛАТЫ С ПРЕМИЕЙ

e2: Имя: Анна, Возраст: 30, Должность: Старший менеджер, Оклад: 60000
Премия: 15%
Итоговая зарплата: 69000

e5: Имя: Тапуа, Возраст: 27, Должность: Cleaner, Оклад: 20000
Итоговая зарплата (с премией 20%): 24000

7. ПРИНЦИП ПОДСТАНОВКИ

Передаём e2 (Employee) в функцию printPersonInfo:
Вызван конструктор копирования Person
Информация о человеке: Имя: Анна, Возраст: 30

```

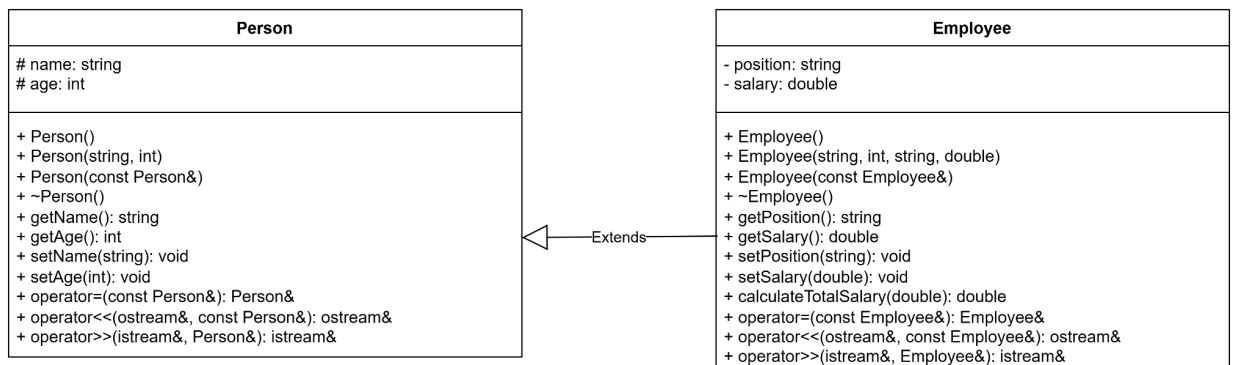
Вызов функции createPerson:
Вызван конструктор с параметрами Person
p5: Имя: Петр, Возраст: 35

8. ВСЕ ОБЪЕКТЫ

p1: Имя: Неизвестно, Возраст: 0
p2: Имя: Иван, Возраст: 25
p3: Имя: Иван, Возраст: 25
p4: Имя: Иван, Возраст: 25
p5: Имя: Петр, Возраст: 35
e1: Имя: Неизвестно, Возраст: 0, Должность: Не определена, Оклад: 0
e2: Имя: Анна, Возраст: 30, Должность: Старший менеджер, Оклад: 60000
e3: Имя: Мария, Возраст: 30, Должность: Менеджер, Оклад: 50000
e4: Имя: Анна, Возраст: 30, Должность: Старший менеджер, Оклад: 60000
e5: Имя: Таня, Возраст: 27, Должность: Cleaner, Оклад: 20000
Вызван деструктор Person для: Петр
Вызван деструктор Employee для: Таня
Вызван деструктор Person для: Таня
Вызван деструктор Employee для: Анна
Вызван деструктор Person для: Анна
Вызван деструктор Person для: Иван
Вызван деструктор Employee для: Мария
Вызван деструктор Person для: Мария
Вызван деструктор Employee для: Анна
Вызван деструктор Person для: Анна
Вызван деструктор Employee для: Неизвестно
Вызван деструктор Person для: Неизвестно
Вызван деструктор Person для: Иван
Вызван деструктор Person для: Иван
Вызван деструктор Person для: Неизвестно

```

UML:



Вопросы:

1. Для чего используется механизм наследования? Наследование - это механизм получения нового класса на основе уже существующего. Существующий класс может быть дополнен или изменен для создания нового класса.
2. Каким образом наследуются компоненты класса, описанные со спецификатором public? public – член класса может использоваться любой

функцией, которая является членом данного или производного класса, а также к public-членам возможен доступ извне через имя объекта.

3. Каким образом наследуются компоненты класса, описанные со спецификатором `private`? `private` – член класса может использоваться только функциями-членами данного класса и функциями-друзьями своего класса. В производном классе он недоступен.

4. Каким образом наследуются компоненты класса, описанные со спецификатором `protected`? `protected` – то же, что и `private`, но дополнительно член класса с данным атрибутом доступа может использоваться функциями-членами и функциями-друзьями классов, производных от данного.

5. Каким образом описывается производный класс? Синтаксис определения производного класса: `class имя_класса : список_базовых_классов { список_компонентов_класса }`;

6. Наследуются ли конструкторы? Поскольку конструкторы не наследуются, при создании производного класса наследуемые им данные-члены должны инициализироваться конструктором базового класса.

7. Наследуются ли деструкторы? В тексте прямо не сказано, но по аналогии с конструкторами и общим принципом: деструкторы не наследуются, однако вызываются автоматически в обратном порядке. В разделе 2.2 говорится: "Уничтожаются объекты в обратном порядке: сначала производный, потом его компоненты-объекты, а потом базовый объект". Это подразумевает, что деструктор производного класса вызывает деструктор базового, но сам деструктор не наследуется.

8. В каком порядке конструируются объекты производных классов? Объекты класса конструируются снизу вверх: сначала базовый, потом компоненты-объекты (если они имеются), а потом сам производный класс.

9. В каком порядке уничтожаются объекты производных классов? Уничтожаются объекты в обратном порядке: сначала производный, потом его компоненты-объекты, а потом базовый объект. Порядок уничтожения объекта противоположен по отношению к порядку его конструирования.

10. Что представляют собой виртуальные функции и механизм позднего связывания? Виртуальные функции предоставляют механизм позднего (отложенного) или динамического связывания. Интерпретация каждого вызова виртуальной функции через указатель на базовый класс зависит от значения этого указателя, т.е. от типа объекта, для которого выполняется вызов. Выбор виртуальной функции зависит от типа объекта, на который фактически направлен указатель, а не от типа указателя.

11. Могут ли быть виртуальными конструкторы? Деструкторы?

Конструкторы не могут быть виртуальными, в отличие от деструкторов.

Практически каждый класс, имеющий виртуальную функцию, должен иметь виртуальный деструктор.

12. Наследуется ли спецификатор `virtual`? Виртуальность наследуется. После того как функция определена как виртуальная, ее повторное определение в производном классе (с тем же самым прототипом) создает в этом классе новую виртуальную функцию, причем спецификатор `virtual` может не использоваться.

13. Открытое наследование устанавливает отношение «is-a» (является разновидностью). Производный класс является специализацией базового, и его объекты могут использоваться везде, где ожидаются объекты базового класса (принцип подстановки). Пример: `Teacher` является `Employee`, а `Employee` является `Student`. Это не отношение «часть-целое» (композиция), а отношение иерархии типов.

14. Какое отношение устанавливает между классами закрытое наследование? Закрытое наследование – это наследование реализации, в этом случае принцип подстановки не соблюдается.

15. В чем заключается принцип подстановки? Принцип подстановки: везде, где может быть использован объект базового класса (при присваивании, при передаче параметров и возврате результата), вместо него разрешается использовать объект производного класса. Данное положение работает и для ссылок и для указателей. Обратное неверно.

16. Объект `x` типа `Teacher` будет содержать в памяти все поля всей цепочки наследования: `age`, `name`, `post`, `stage`. Однако доступ к ним различается:

- `name` — доступен публично (`public`)
- `post`, `stage` — доступны внутри классов-наследников (`protected`)
- `age` — присутствует в объекте, но скрыт (`private`), поэтому прямой доступ `x.age` запрещён. Получить/изменить его можно только через публичные методы класса `Student`.

17. Для классов `Student`, `Employee` и `Teacher` написать конструкторы без параметров.

```
Student() : age(0), name("") {}
```

```
Employee() : Student(), post("") {}
```

```
// Student() вызовется автоматически, инициализируя name и age
```

```
Teacher() : Employee(), stage(0) {}
```

// Employee() вызовется, затем Student()

18. Для классов Student, Employee и Teacher написать конструкторы с параметрами.

```
Student(int a, string n) : age(a), name(n) {}
```

```
Employee(string n, string p) : Student(0, n), post(p) {}
```

// Вызываем конструктор Student, передаём имя, возраст ставим по умолчанию или добавляем параметр a

```
Teacher(string n, string p, int s) : Employee(n, p), stage(s) {}
```

// Вызываем конструктор Employee, который вызовет Student

19. Для классов Student, Employee и Teacher написать конструкторы копирования.

```
Student(const Student& other) : age(other.age), name(other.name) {}
```

```
Employee(const Employee& other) : Student(other), post(other.post) {}
```

// Student(other) копирует поля базового класса

```
Teacher(const Teacher& other) : Employee(other), stage(other.stage) {}
```

// Employee(other) копирует поля Employee и Student

20. Для классов Student, Employee и Teacher определить операцию присваивания.

```
Student& operator=(const Student& s) {
```

```
    if (&s == this) return *this;
```

```
    name = s.name;
```

```
    age = s.age;
```

```
    return *this;
```

```
}
```

```
Employee& operator=(const Employee& e) {
```

```
    if (&e == this) return *this;
```

```
    Student::operator=(e);
```

```
    post = e.post;
```

```
    return *this;
```

```
}
```

```
Teacher& operator=(const Teacher& t) {  
    if (&t == this) return *this;  
    Employee::operator=(t);  
    stage = t.stage;  
    return *this;  
}
```